SL2DT-03: Log Ingest Architecture

> **Series:** SL2DT — Sumo Logic to Dynatrace | **Notebook:** 3 of 11 | **Created:** April 2026 | **Last Updated:** 07/20/2026

Overview

Goal of this step: stand up Dynatrace ingest that produces parity with (or improvement over) the Sumo collector footprint. Design Grail buckets, deploy OneAgent + OTel + OpenPipeline pipelines, convert every Field Extraction Rule into an OpenPipeline processor.

This step is gating. Downstream monitors (SL2DT-05), dashboards (SL2DT-06), and the translation pass (SL2DT-04) all assume logs are flowing into the correct buckets with the expected fields. Don't start those until the ingest layer produces at least 5–10% of final production volume for validation.

> **Metrics are a separate workstream.** This notebook covers the **log** ingest path. If the source account also collects metrics — Telegraf, hosted metrics sources, StatsD, or Prometheus scrape — run **SL2DT-10: Migrating Telegraf-Collected Metrics** alongside this step rather than after cutover.

Table of Contents

Prerequisites

Requirement	Details
Audience	Platform engineering + ingest team
Dynatrace tenant	Gen3 SaaS, Platform Token with `storage:buckets:write`, `settings:objects:write`, `openpipeline:configurations:write` scopes
Prior reading	SL2DT-02 (taxonomy map + collector inventory + FER inventory); OPMIG-01 (OpenPipeline fundamentals); K8S-01 (if K8s sources)
Tools	Terraform + Monaco for config-as-code (recommended); kubectl for K8s; cURL for Platform Token validation

1. What You'll Produce

Artifact	Purpose
Grail bucket configs	One per retention/IAM tier from taxonomy map
OneAgent rollout config	Hosts auto-instrumented
OTel Collector manifests	Non-OneAgent sources (syslog, HTTP, cloud)
OpenPipeline pipeline configs	Per bucket + per `_sourceCategory` scope
OpenPipeline processors	FER equivalents (one per original FER)
Ingest validation report	Parity check: Sumo volume vs DT volume per scope

Commit to `config/ingest/` as Terraform HCL + Monaco YAML.

2. Prerequisites & Access

This step uses two distinct credential types — they are **not interchangeable**, and mixing them up is a common source of "permission denied" confusion during ingest setup.

Platform Token — query/runtime API calls

Platform Token (prefix `dt0s16.`) with these scopes, for runtime/query-time API calls (bucket read/write of data, log/settings read/write, OpenPipeline config read/write, IAM policy read/write via the platform APIs):

```
storage:buckets:read, storage:buckets:write
storage:logs:read, storage:logs:write
settings:objects:read, settings:objects:write
openpipeline:configurations:read, openpipeline:configurations:write
iam:policies:read, iam:policies:write (for bucket-scoped policies)
```

Critical: Platform Token uses `Authorization: Bearer <token>`, not `Api-Token`. See user memory if needed.

```
export DT_TENANT="https://&lt;env-id&gt;.apps.dynatrace.com"
export DT_TOKEN="dt0s16.XXXXX..."
curl -H "Authorization: Bearer $DT_TOKEN"
"$DT_TENANT/platform/classic/environment-api/v2/tokens/lookup" \
  -H "Content-Type: application/json" -d "{\"token\": \"$DT_TOKEN\"}"
```

OAuth client — Terraform-managed bucket definitions

The `dynatrace_platform_bucket` Terraform resource used later in this notebook is OAuth-client-only — a Platform Token cannot drive it, regardless of the `storage:*` scopes above. Provision an OAuth client with:

```
storage:bucket-definitions:read    (View bucket metadata)
storage:bucket-definitions:write   (Write buckets)
storage:bucket-definitions:delete  (Delete buckets)
```

```
export DT_CLIENT_ID="dt0s02.XXXXX"
export DT_CLIENT_SECRET="dt0s02.XXXXX.YYYYY"
export DT_ACCOUNT_ID="&lt;account-uuid&gt;"
```

The Platform Token scopes above (`storage:buckets:*`) gate query/read access to bucket **data** at runtime — they are a separate concern from OAuth-client-gated bucket **definition** CRUD via Terraform.

> This OAuth-client-only requirement isn't unique to buckets — it applies to every IAM resource too, and **AUTOM-04**'s consolidated resource catalog now documents the full list of OAuth-client-only resources in one place (buckets, IAM group/policy/bindings) alongside the Platform-Token-eligible ones.

Terraform + Monaco

```
# Terraform
terraform init

# Monaco (for bulk config promotion)
monaco --version
```

3. Grail Bucket Design

From the taxonomy map (SL2DT-02 output), define buckets.

Bucket Naming Convention

```
{env}_{purpose}[_{scope}]
```

Examples:
- `custom_logs_prod` — production logs, default
- `custom_logs_prod_db` — production database logs (separate retention)
- `custom_logs_preprod` — pre-production
- `custom_logs_dev` — development
- `audit_logs` — audit trail (compliance retention)

Settings 2.0 Schema

Buckets are managed via the Dynatrace Settings API using schema `builtin:bucket-configuration` :

```
{
  "schemaId": "builtin:bucket-configuration",
  "scope": "environment",
  "value": {
    "name": "custom_logs_prod",
    "displayName": "Production Logs",
    "retention": 90,
    "table": "logs"
  }
}
```

Terraform Example

```
resource "dynatrace_platform_bucket" "custom_logs_prod" {
  name         = "custom_logs_prod"
  display_name = "Production Logs"
  retention    = 90
  table        = "logs"
}

resource "dynatrace_platform_bucket" "custom_logs_prod_db" {
  name         = "custom_logs_prod_db"
  display_name = "Production Database Logs"
  retention    = 180
  table        = "logs"
}

resource "dynatrace_platform_bucket" "audit_logs" {
  name         = "audit_logs"
  display_name = "Audit Trail"
  retention    = 365
  table        = "logs"
}
```

Validate buckets exist

```
// Confirm bucket inventory
fetch dt.system.buckets, from:-5m
| fields name = `name`, displayName = `displayName`, retention = `retention`
| sort name asc
```

4. `_sourceCategory` Mapping — Implementation

Two decisions come out of the taxonomy map (SL2DT-02): bucket routing and attribute preservation. Both get implemented in OpenPipeline.

Pattern A — Bucket routing at ingest

Route based on the source identifier (hostname pattern, tag, container label, etc.):

```
{
  "pipelineId": "pipeline_prod_routing",
  "matchers": [
    { "matcher": "k8s.namespace.name == 'prod'" },
    { "matcher": "log.source startsWith 'prod/'" }
  ],
  "targetBucket": "custom_logs_prod"
}
```

Pattern B — Attribute preservation

Add a `dt.source_entity` field that mirrors the Sumo `_sourceCategory` value so queries can still group by it:

```
{
  "processor": "add-field",
  "field": "dt.source_entity",
  "expression": "extract(log.source, 'regex_for_sourcecategory_equivalent')"
}
```

Most customers do both: bucket for isolation, attribute for query-time grouping.

Query what you get

After OpenPipeline is live, this should return your taxonomy:

```
// Inspect the preserved _sourceCategory equivalent
fetch logs, from:-1h, bucket:"custom_logs_prod"
| summarize c = count(), by:{dt.source_entity}
| sort c desc
| limit 50
```

5. Deploy OneAgent for Host Collectors

Every Sumo Installed Collector has a OneAgent equivalent. For hosts already running a Sumo Installed Collector:

Linux / Windows Hosts

```
# Fetch installer from tenant UI or API, then:
/bin/sh Dynatrace-OneAgent-Linux-1.xxx.sh \
  --set-app-log-content-access=true \
  --set-host-group=prod-web

# Verify
oneagentctl --get-host-tag
oneagentctl --get-logmonitoring-enabled
```

AWS / Azure / GCP Hosts

Use Dynatrace Clouds app (AWS) or cloud-native integrations. No manual OneAgent install.

Kubernetes

Install the Dynatrace Operator, then configure DynaKube:

```
apiVersion: dynatrace.com/v1beta6
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  apiUrl: https://&lt;env-id&gt;.apps.dynatrace.com/api
  tokens: tokens-secret
  oneAgent:
    cloudNativeFullStack:
      tolerations: []
      nodeSelector: {}
  activeGate:
    capabilities:
      - kubernetes-monitoring
      - routing
  logMonitoring:
    ingestRuleMatchers:
      - attribute: k8s.namespace.name
        operator: matches
        values: ["prod-*", "preprod-*"]
```

Capture host groupings

Tag hosts per team/environment. These tags feed the bucket routing in OpenPipeline.

```
oneagentctl --set-host-tag="team=payments"
oneagentctl --set-host-tag="env=prod"
```

Verify ingest

```
// Confirm OneAgent-instrumented hosts are sending logs
fetch logs, from:-15m
| filter isNotNull(dt.entity.host)
| summarize c = count(), by:{entityName(dt.entity.host)}
| sort c desc
| limit 20
```

6. Deploy OTel Collector for Non-OneAgent Sources

For sources where OneAgent doesn't apply (syslog listener, HTTP upload, message-queue consumer, proprietary agents), deploy an OTel Collector.

Minimal OTel Collector Config

```
receivers:
  syslog:
    tcp:
      listen_address: 0.0.0.0:5514
    protocol: rfc5424
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317

processors:
  resourcedetection:
    detectors: [env, system]
  batch:
    send_batch_size: 1000
    timeout: 5s

exporters:
  otlphttp:
    endpoint: https://&lt;env-id&gt;.live.dynatrace.com/api/v2/otlp
    headers:
      Authorization: "Api-Token dt0c01.XXX"

service:
  pipelines:
    logs:
      receivers: [syslog, otlp]
      processors: [resourcedetection, batch]
      exporters: [otlphttp]
```

Note on auth: OTel HTTP exporter pairs with Classic API Token (`dt0c01.`) + `Api-Token` scheme for the `/api/v2/otlp` endpoint. Platform Token is not used for OTel ingest auth as of April 2026.

HTTP Source Equivalent

Sumo "Hosted HTTP Source" → OpenPipeline HTTP endpoint (no OTel in between):

```
curl -X POST "$DT_TENANT/api/v2/logs/ingest" \
  -H "Authorization: Api-Token dt0c01.XXX" \
  -H "Content-Type: application/json" \
  -d '[{"content": "app log line", "log.source": "prod/api/web"}]'
```

Scripted ingestion from CloudWatch, EventHub, Pub/Sub — replace with Dynatrace cloud integrations (AWS Clouds app, Azure, GCP) rather than dual-writing via HTTP.

7. Configure OpenPipeline Pipelines

For each bucket, define a pipeline that:

1. **Matches** — which logs belong here
2. **Processes** — parse, enrich, redact
3. **Routes** — to the target bucket

Example: Production API Logs Pipeline

Note on API surface: the OpenPipeline **Configurations API** (the older `pipelineId` / `matchers` / `processors` JSON shape) is deprecated with an end-of-life of **June 29, 2026**. Current pipeline configuration goes through the **Settings API**, using the per-scope schema family `builtin:openpipeline.<scope>.pipelines` (plus `.routing` and `.ingest-sources` for matching/routing and source config), or the equivalent Terraform resource. Both forms below reflect the current shape.

Settings API — `builtin:openpipeline.logs.pipelines`:

```
{
  "schemaId": "builtin:openpipeline.logs.pipelines",
  "scope": "environment",
  "value": {
    "displayName": "Production API Logs",
    "customId": "pipeline_prod_api",
    "processing": {
      "processors": [
        {
          "type": "dql",
          "id": "processor_parse_http_fields",
          "description": "Parse method/path/status from content",
          "matcher": "true",
          "dql": {
            "script": "parse content, \"LD 'method=' WORD:http.method '
path=' DATA:http.path ' status=' INT:http.status\""
          },
          "enabled": true
        },
        {
          "type": "fieldsAdd",
          "id": "processor_add_source_entity",
          "description": "Derive dt.source_entity from k8s.deployment.name",
          "matcher": "true",
          "fieldsAdd": {
            "fields": [
              { "name": "dt.source_entity", "value": "concat('prod/api/',
k8s.deployment.name)" }
            ]
          },
          "enabled": true
        }
      ]
    }
  }
}
```

Terraform equivalent — `dynatrace_openpipeline_v2_logs_pipelines`:

```
resource "dynatrace_openpipeline_v2_logs_pipelines" "pipeline_prod_api" {
  display_name = "Production API Logs"
  custom_id    = "pipeline_prod_api"
  processing {
    processors {
      processor {
        type        = "dql"
        id          = "processor_parse_http_fields"
        description = "Parse method/path/status from content"
        matcher     = "true"
        dql {
          script = "parse content, \"LD 'method=' WORD:http.method ' path='
DATA:http.path ' status=' INT:http.status\""
        }
        enabled = true
      }
      processor {
        type        = "fieldsAdd"
        id          = "processor_add_source_entity"
        description = "Derive dt.source_entity from k8s.deployment.name"
        matcher     = "true"
        fields_add {
          fields {
            field {
              name  = "dt.source_entity"
              value = "concat('prod/api/', k8s.deployment.name)"
            }
          }
        }
        enabled = true
      }
    }
  }
}
```

Redaction (masking the 16-digit card-number pattern from the earlier illustrative example) belongs in a dedicated masking/DQL processor stage — see OPLOGS-03 and OPIPE for the current redaction processor shape rather than a standalone `"type": "redact"` block, which does not exist in the current API.

Route-to-bucket is a separate concern from the pipeline's processors — it's handled by the pipeline's `storage` stage / bucket-assignment processor (`bucket_assignment_processor` in Terraform), not a top-level `targetBucket` field on the pipeline object.

Validation Queries

```
// Check that parsing produces the expected fields
fetch logs, from:-15m, bucket:"custom_logs_prod"
| filter dt.source_entity == "prod/api/payments"
| filter isNotNull(http.status)
| summarize c = count(), by:{http.method, http.status}
| sort c desc
```

8. Convert FERs to OpenPipeline Processors

Every Sumo FER becomes one OpenPipeline parse processor. The logic is the same; only the pattern syntax changes.

Sumo FER Example

```
# FER name: api-parse-req
# Scope: _sourceCategory=prod/api/*
# Parse expression:
"method=*" "path=*" "status=*"
```

Fields extracted: `method`, `path`, `status`.

Dynatrace OpenPipeline Equivalent

```
{
  "type": "parse",
  "matchers": [{ "attribute": "dt.source_entity", "operator": "startsWith",
"value": "prod/api/" }],
  "pattern": "LD 'method=' DATA:http.method ' path=' DATA:http.path '
status=' INT:http.status"
}
```

DPL Pattern Cheat Sheet

Sumo regex	DPL matcher
`.+` (greedy)	`DATA` (lazy, default)
`\w+` (word chars)	`WORD`
`\d+` (digits)	`INT`
`\d+\.\d+` (float)	`DOUBLE`
`\S+` (non-space)	`LD`
`\d+\.\d+\.\d+\.\d+` (IPv4)	`IPADDR`
ISO 8601 timestamp	`TIMESTAMP`
Quoted string	`QUOTED_STRING`
`\{.*\}` (JSON blob)	`JSON`

FER Conversion Workflow

1. For each FER in `inventory/fers.json`, identify the scope.
2. Map the parse expression to DPL matchers.
3. Create the OpenPipeline processor on the matching pipeline.
4. Validate extraction via a DQL query.

Check: Parsed fields are available

```
// After FER conversion, confirm extraction is working
fetch logs, from:-15m, bucket:"custom_logs_prod"
| filter startsWith(dt.source_entity, "prod/api/")
| filter isNotNull(http.method)
| limit 10
| fields content, http.method, http.path, http.status
```

9. Validate Ingest Parity

Parity = volume per scope per hour in DT is within 5% of Sumo volume for the same scope.

Sumo-side baseline

```
*
| count by _sourceCategory
| where _count &gt; 0
```

Dynatrace-side comparison

```
// Count per scope in DT — compare to Sumo baseline
fetch logs, from:-1h
| summarize c = count(), by:{dt.source_entity}
| sort c desc
| limit 50
```

Parity report

`_sourceCategory`	Sumo count/hr	DT count/hr	% diff	Status
prod/api/payments	18,452	18,110	-1.9%	✅
prod/web/frontend	9,300	9,044	-2.8%	✅
prod/db/primary	2,103	1,845	-12.3%	⚠️ investigate
…	…	…	…	…

Target: ≥95% of scopes within 5% diff. Flag any scope with >10% diff for investigation before proceeding.

Common Parity Gaps

Symptom	Likely Cause	Fix
DT count much lower	OneAgent or OTel not deployed to all sources	Check deployment inventory
DT count much higher	Duplicate ingest paths (Sumo still writing + DT ingesting same source twice)	Audit collector config
DT missing entire scope	`_sourceCategory` pattern didn't match any OpenPipeline matcher	Adjust matcher
Parsed fields missing	FER processor not applied to the pipeline	Re-check processor scope

10. Step Exit Criteria

G3 — Ingest Parity Achieved

- [ ] All planned buckets exist and retention configured
- [ ] OneAgent deployed to 100% of host-based collectors
- [ ] OTel Collector deployed for non-OneAgent sources
- [ ] OpenPipeline pipelines configured per bucket
- [ ] Every FER has a corresponding OpenPipeline processor
- [ ] `dt.source_entity` attribute populated on all logs
- [ ] Parity report: ≥95% of scopes within 5% of Sumo volume
- [ ] No scope with >10% volume gap uninvestigated

Next step: SL2DT-04 — SumoQL → DQL Translation (translate the query surface with the `sumoql-to-dql` skill).

---

11. References

Log ingest paths

- Ingest from (DT docs)

- Dynatrace OneAgent (DT docs)
- OpenTelemetry ingest (DT docs)
- Logs (DT docs)
- Dynatrace API (DT docs)

Routing, parsing, and storage

- OpenPipeline (DT docs)
- DPL Pattern Reference (DT docs)
- Grail (DT docs)
